

From a Sphere to a Coupled Planetary Renderer

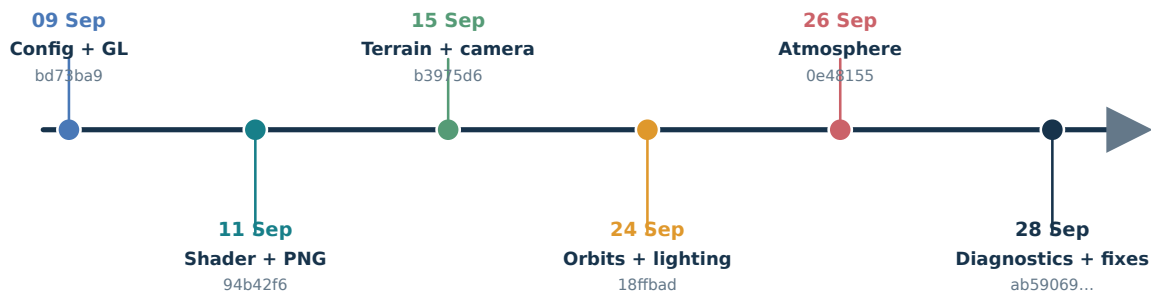
A chronological engineering journal of PlanetSimulation

Repository study · 9–28 September 2026 · version 0.0.1

Abstract. This paper reconstructs the development of PlanetSimulation from its Git history, source code, tests, and versioned image captures. The implementation progressed from a configurable OpenGL sphere to a terrain-bearing, hierarchical orbital scene with reflected illumination, terrain shadows, atmospheric scattering and refraction, and interactive diagnostics. We explain the numerical and rendering choices that made each stage possible: deterministic height sampling on an adaptive icosahedral mesh, absolute-time Kepler evaluation with barycentric recoil, linear-light composition, density-dependent atmospheric integration, and latency-aware quality control. Thirteen generated vector diagrams expose the algorithms; repository screenshots show actual rendering. A historical three-stage GPU benchmark is reproduced with its original hardware qualification. The final repository contains 39 automated tests, including headless OpenGL and input checks. The system remains an intentionally compact simulation: its orbital hierarchy is prescribed rather than an N-body integrator, and its 20 FPS controller is best-effort.

Keywords: real-time graphics · procedural terrain · Keplerian dynamics · atmospheric optics · adaptive rendering · reproducible engineering

A chronological reconstruction from selected commits



Chronology follows code milestones; each point anchors a group of commits, not an isolated invention.

Figure 1: Selected historical anchors. The source history contains 171 commits through 9661b8e; the nodes group related changes rather than claiming a one-commit implementation. This figure is a generated schematic.

1. Scope and method

This is an artifact study, not a claim that every rendered effect was invented here. The ordering follows `git log --reverse` from the initial commit 5a23a47 to 9661b8e. Commit identifiers below identify implementation checkpoints; formulas are checked against the current source, and the discussion notes where a later correction changed an earlier design. Rendering images are taken from `docs/screenshots/`, whose generation manifest records commands, hashes, renderer, and UTC timestamps. Diagrams in `figures/` are explanatory reconstructions generated by `make_figures.py`; they are not captured output.

The chronology has six engineering phases: (i) configuration and a reliable graphics pipeline, (ii) planet-local terrain and navigation, (iii) hierarchical orbits and lighting, (iv) atmosphere and performance engineering, (v) interactive diagnostics, and (vi) targeted correctness fixes. Figure 1 gives the sequence. The paper treats validation as part of each phase: visual plausibility alone cannot establish a matrix convention, preserve a stable surface during walking, or prove that a shadowed ocean receives no direct reflection.

2. 9–13 September: a testable rendering foundation

2.1. From configuration to a projected sphere

The initial build and math abstractions (1385b74) were followed by structured JSON scenario parsing (bd73ba9) and a GLFW/OpenGL loop (c96f609). The first visible output was a camera-framed Sun and planet (b87d5af), then a shader/mesh pipeline (8246a31). A position in a planet’s local mesh is transformed by a model matrix, a camera view matrix, and a projection matrix. In the project’s eventual column-vector convention,

$$x_{\text{clip}} = PVMx_{\text{local}}, \quad x_{\text{ndc}} = \frac{x_{\text{clip}}}{w_{\text{clip}}}.$$

This equation was operationally significant: the early history contains repeated fixes to cross-product order, handedness, matrix storage, and multiplication indices. Tests eventually asserted behavior rather than literal matrix entries; on 15 September, ee48919 replaced the custom `Matrix4` and `Vector3` implementation with GLM. That reduced the number of independent conventions the project had to maintain. Figure 2 records the resulting data path.

From a configured body to a framebuffer pixel

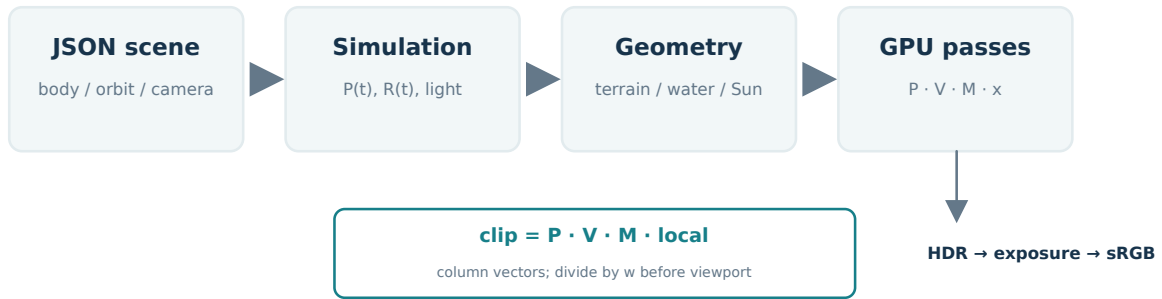


Figure 2: The foundational transform and shading path. $P \cdot V \cdot M$ follows the current shader convention; the final display transform happens after linear-light composition. Generated schematic.

A hidden-window `--render-test` and PNG smoke test appeared in 94b42f6. That made camera framing, framebuffer readback, and image output observable in CI-like environments. The writer evolved through several early PNG attempts and now resides in `src/rendering/diagnostics/PngWriter.h`, using libpng. Figure 3 is an actual headless capture of the compact regression scene. It should be read as evidence of the current renderer at a fixed scene, not as a screenshot of the 2026-09-11 binary.



Figure 3: Current headless rendering of the frozen compact scenario. It preserves the early scientific question—*are the bodies positioned and projected correctly?*—while using the present version’s renderer. Source: `docs/screenshots/solar-view.png`; provenance: `docs/captures/generation.json`.

2.2. A reproducible scene as an experimental unit

The JSON parser performs type and range checks before GPU resources are replaced. Later scene replay stores a resolved scenario, camera, time, and quality choices next to a PNG. This is more useful than a screenshot alone: the image is an observation, while the sidecar specifies a repeatable input state. On live reload, a new CPU scene is staged before the current one is exchanged, so an invalid or half-written file retains the last valid scene. The archive therefore separates three kinds of evidence: source history, executable tests, and image captures with replay metadata.

3. 14–16 September: terrain and navigation on a rotating sphere

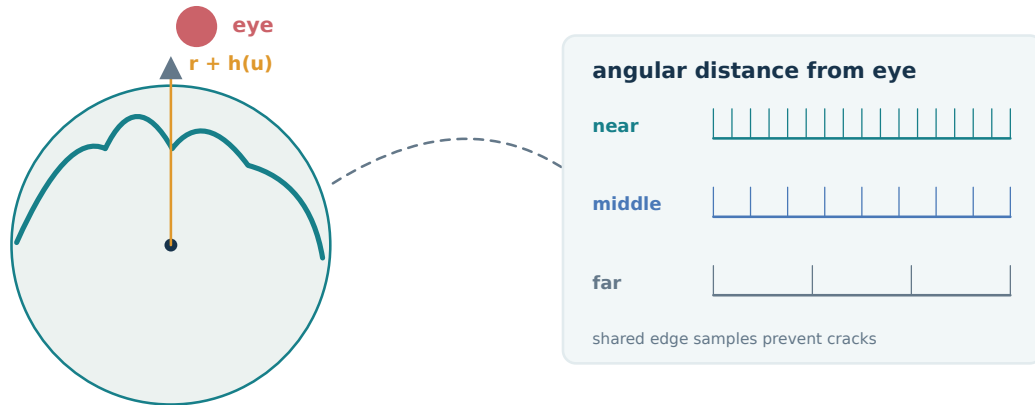
3.1. Procedural surface as a camera-independent function

The 15 September sequence `1f8a07a` through `3d73170` introduced and refined noise-driven planet generation. The height function is evaluated in a normalized planet-local radial direction u , not in camera coordinates. A simplified form is

$$h(u) = h_{\text{landscape}(u)} + \sum_i A_i f_i(u; \text{seed}_i, \text{frequency}_i, \text{octaves}_i).$$

The implemented functions include value-noise fBm and a ridged variant. Broad continent and cliff fields determine large-scale shape; finer octaves add local detail. The seed and local coordinate frame make repeated samples of the same point deterministic. This matters because terrain geometry is rebuilt as the camera moves: if h changed with view distance, the ground itself would rise and fall. The current code explicitly samples `heightAt(radial)` during every LOD rebuild. Figure 4 distinguishes the fixed field from its variable triangulation.

Stable height field and camera-dependent tessellation



(u) is fixed in planet-local coordinates; only mesh density follows the camera.

Figure 4: The height field is fixed in planet coordinates; near, middle, and far zones change only sampling density. Bars are illustrative edge densities, not an image of the actual mesh. Generated schematic.

The sphere begins with 20 icosahedral faces, subdivided twice into 320 stable base faces. For nearby views, each base face is assigned one of three edge-segment zones using its angular distance to the eye. A previous-zone hysteresis margin prevents repeated switching at a threshold. Steep faces can receive extra segments if the triangle budget permits. Edges shared by unequal zones are sampled once at the finer requirement, then reused on both sides; concentric interior rings fill the faces without T-junctions. The budget loop first removes optional steep refinement and then lowers less-important zones. This is an engineering compromise between stable walking geometry, crack-free transitions, and a hard per-planet triangle ceiling.

The sampled height feeds normals and terrain color factors as well as eye clearance. Water is a separate spherical shell. The local camera follows the planet's translation and rotation through a planet-fixed frame, so the same latitude/longitude refers to the same terrain location after orbital motion or spin. Figure 5 shows the current rendered outcome rather than an intermediate development snapshot.

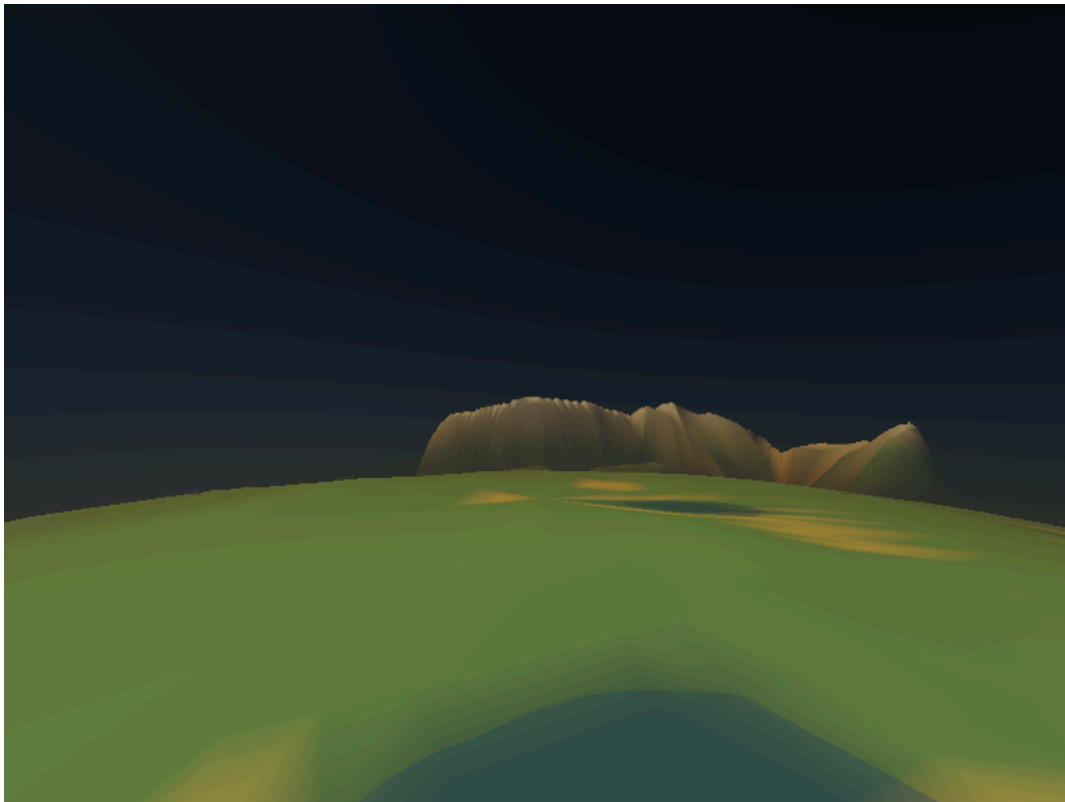


Figure 5: Current surface-camera capture of procedurally generated terrain. The geometry derives from the stable height function while its tessellation is selected for this camera. Source: current gallery.

4. 24 September: absolute-time dynamics and illumination

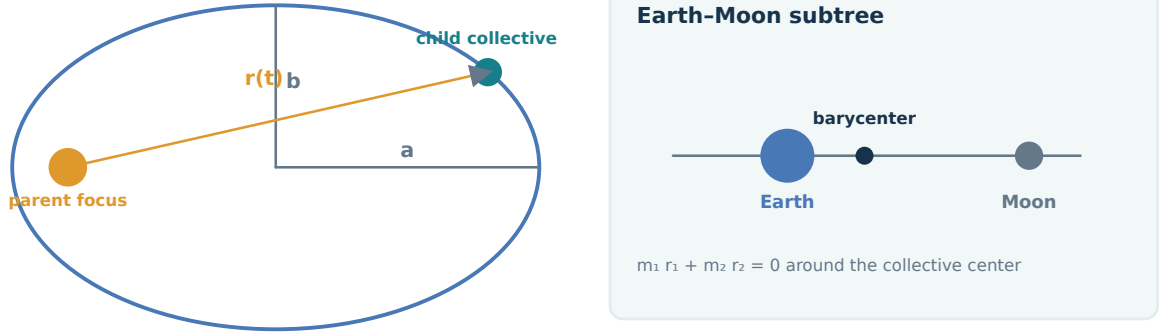
4.1. Prescribed Kepler ellipses with collective mass

The major simulation change **18ffb6d** replaced decorative orbital speed metadata with a hierarchy of Kepler ellipses. A body names its parent and supplies semi-major and semi-minor axes a, b , a mean anomaly, inclination, ascending node, and periapsis orientation. Eccentricity and period follow

$$e = \sqrt{1 - \frac{b^2}{a^2}}, \quad \mu = G(m_{\text{parent}} + m_{\text{collective}}), \quad T = 2\pi\sqrt{\frac{a^3}{\mu}}.$$

Here a and b are converted from the scenario's distance unit to metres before applying the SI gravitational constant; the code performs the equivalent calculation in world units. At time t , mean anomaly advances linearly. A bounded bracketing solve obtains eccentric anomaly E from $M = E - e \sin E$, and the orbital-plane position is $(a(\cos E - e), b \sin E)$. Three rotations orient that ellipse in the world. The current implementation uses a rationalized form near extreme periapsis to avoid cancellation. Evaluating at absolute time, rather than integrating velocity frame by frame, avoids cumulative drift and makes pausing, replay, and speed changes deterministic.

Kepler ellipse and hierarchical barycentric recoil



Solve $E - e \sin E = M(t)$; then rotate the orbital-plane ellipse into world space.

Figure 6: An ellipse is specified relative to a focus; a child subtree and its parent recoil around a mass-weighted center. The right panel abstracts the Earth–Moon relation; sizes and offsets are not to scale. Generated schematic.

The hierarchy is not a full N-body solver. Each subtree acts as a collective point mass in its parent's prescribed ellipse; children induce mass-weighted recoil of the parent. Thus an Earth–Moon pair has an internal barycenter, and the Sun moves in response to its descendants while its configured position remains the time-zero anchor. The invariants to test are mass-weighted center of mass, finite positions at high eccentricity, and phase consistency across different frame cadences. Inter-branch perturbations and tidal evolution remain outside the model.

For an **0** overlay added later, the application samples the complete hierarchy at 321 times for each body: ten revolutions at 32 segments per revolution. A moon's inertial trail must therefore inherit its parent's translation rather than drawing one stationary local ellipse. Figure 7 previews this later visualization while making the distinction explicit.

Ten future moon revolutions inherit parent translation

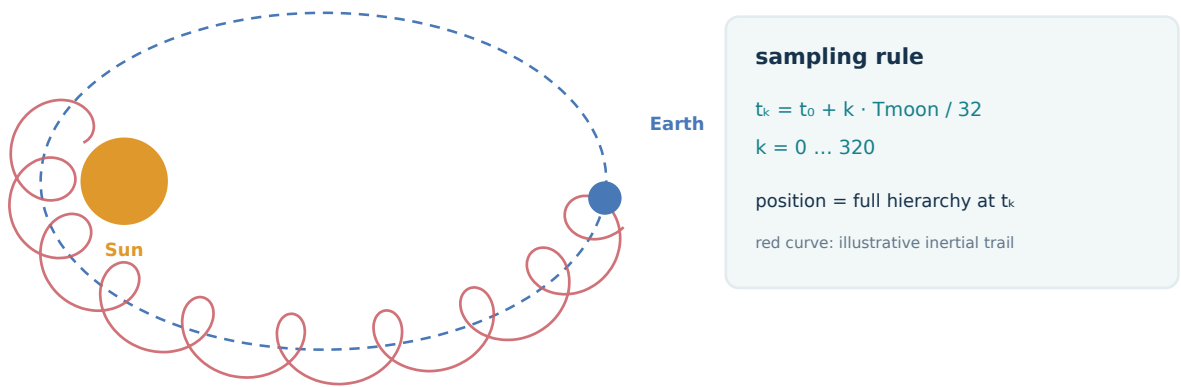


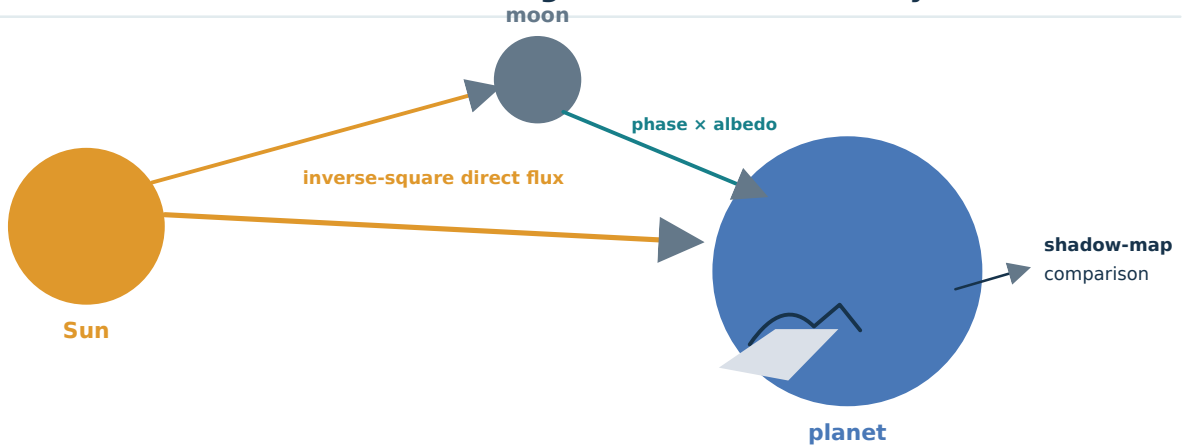
Figure 7: Illustrative future moon trail in the inertial world frame. The repeated local cycles travel with the parent; the curve is explanatory and does not plot a measured scenario trajectory. Generated schematic.

4.2. Light transport before atmospheric composition

01c0b70 introduced moonlight; 96e2014 added terrain shadows; 379544a connected exposure to scene illumination. Per-frame CPU lighting computes direct flux with inverse-square distance and reflected flux with geometric albedo, a diffuse-sphere phase function, and inverse-square reflector-to-receiver falloff. Sun luminosity is derived from absolute bolometric magnitude using the IAU zero point. Shaders sum direct, reflected, and indirect contributions in linear RGB before an exponential display shoulder and sRGB encoding. This ordering is essential: summing display-encoded values would change the physical meaning of each source.

A directional orthographic depth map is made per planet from the local Sun direction. The same map can constrain opaque terrain, water, and later direct atmospheric scattering. The map is cached until local Sun direction, terrain revision, extent, or resolution changes enough to alter its result. Figure 8 separates the direct and reflected paths and the visibility query.

Direct illumination, reflected moonlight and terrain visibility



The same terrain depth map gates opaque shading, water reflection and direct atmospheric scattering.

Figure 8: The Sun contributes direct flux to each body, the Moon reflects a phase-dependent portion toward the planet, and terrain visibility controls direct light. The diagram is schematic; arrow widths do not encode flux.

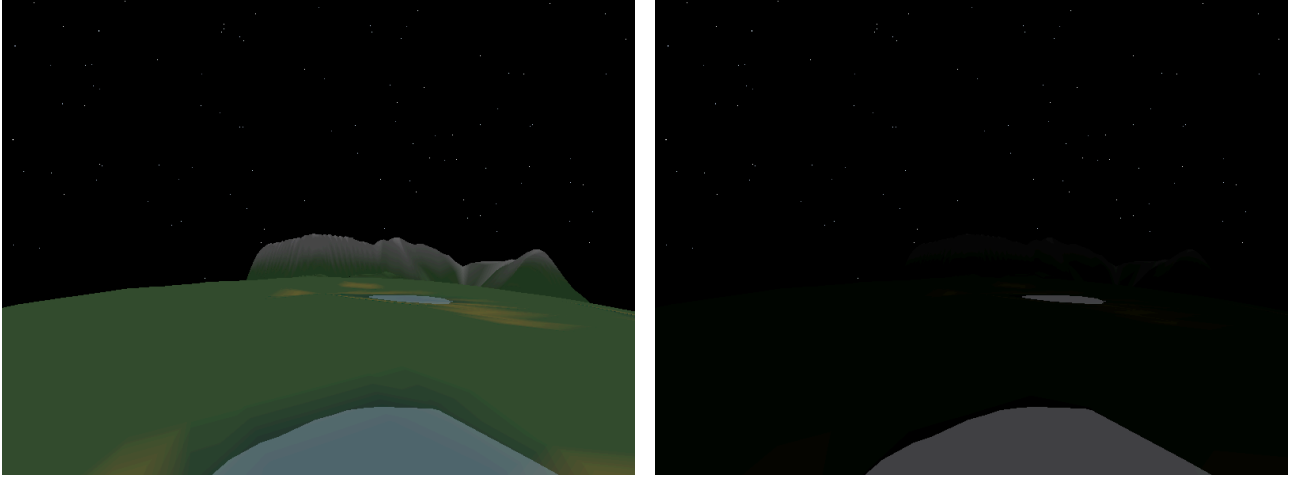


Figure 9: A controlled night-scene comparison from the current gallery: moonlit (left) and moonless (right). The saved replays preserve camera and lighting inputs, allowing the reflected-light term to be isolated.

5. 26 September: atmospheric optics and performance engineering

5.1. Optical state and curved view rays

0e48155 added the atmosphere's composition, density, scattering, and scene composition. For altitude h , the two optical populations use exponential profiles, schematically $\rho_m(h) = \exp\left(-\frac{h}{H_m}\right)$ for molecules and $\rho_a(h) = \exp\left(-\frac{h}{H_a}\right)$ for aerosols. Pressure, temperature, and gas composition determine an approximate visible-band refractivity and Rayleigh coefficients; dust and condensed water add aerosol extinction. The RGB single-scattering source at a view-path sample combines incoming sunlight, phase functions, local density, and transmittance. The direct term now also queries terrain shadow visibility. This is a compact runtime approximation to the scattering formulation in Bruneton and Neyret's work (R2), not their full precomputed multiple-scattering solution.

The configured `temperature_k` enters the reference air state. At fixed pressure, lower temperature raises density and the modeled refractive index. A midpoint integrator bends a view ray according to the perpendicular refractive-index gradient,

$$\frac{d(d)}{ds} = \frac{\nabla n - (d)((d) \cdot \nabla n)}{n}.$$

The shell index tapers continuously to vacuum at its boundary. CPU tests check ray reciprocity and the spherical Snell invariant $nr \sin \theta$; GPU tests compare reprojected sky displacement with the CPU reference. The later commit 9661b8e added direct cold-versus-warm JSON regressions. Figure 10 separates the geometric ray from atmospheric contribution.

Atmospheric density, curved sightline and scattering

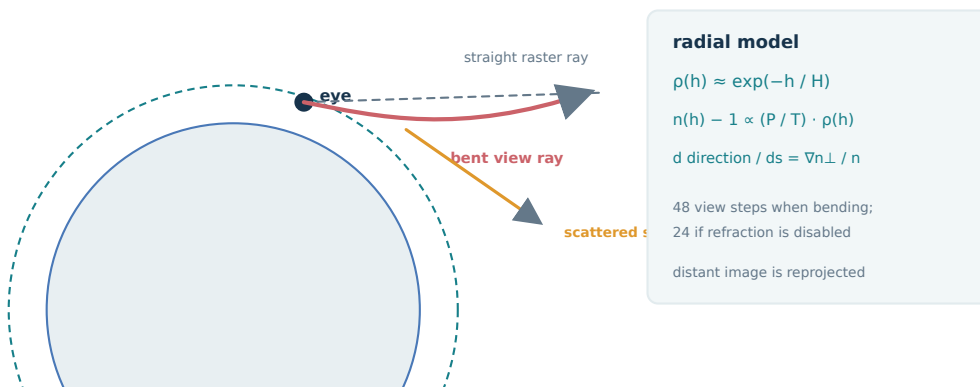


Figure 10: A refractive view ray bends through denser air while solar light contributes scattered radiance. Near terrain keeps its rasterized silhouette; only visible distant imagery is reprojected. Generated schematic.

The current implementation does not bend terrain geometry or direct-light shadow rays. A curved ray that escapes the shell may reproject a visible distant image pixel; an off-screen or hidden source cannot be recon-

structured. Trapped rays do not leak background light. These choices preserve depth and body identity near opaque surfaces while limiting the integration cost. The underlying refractive-index dependence on atmospheric state is well established in metrology (R3), but the constants and miniature-world scale here are an artistic rendering model, not a precision terrestrial atmosphere.

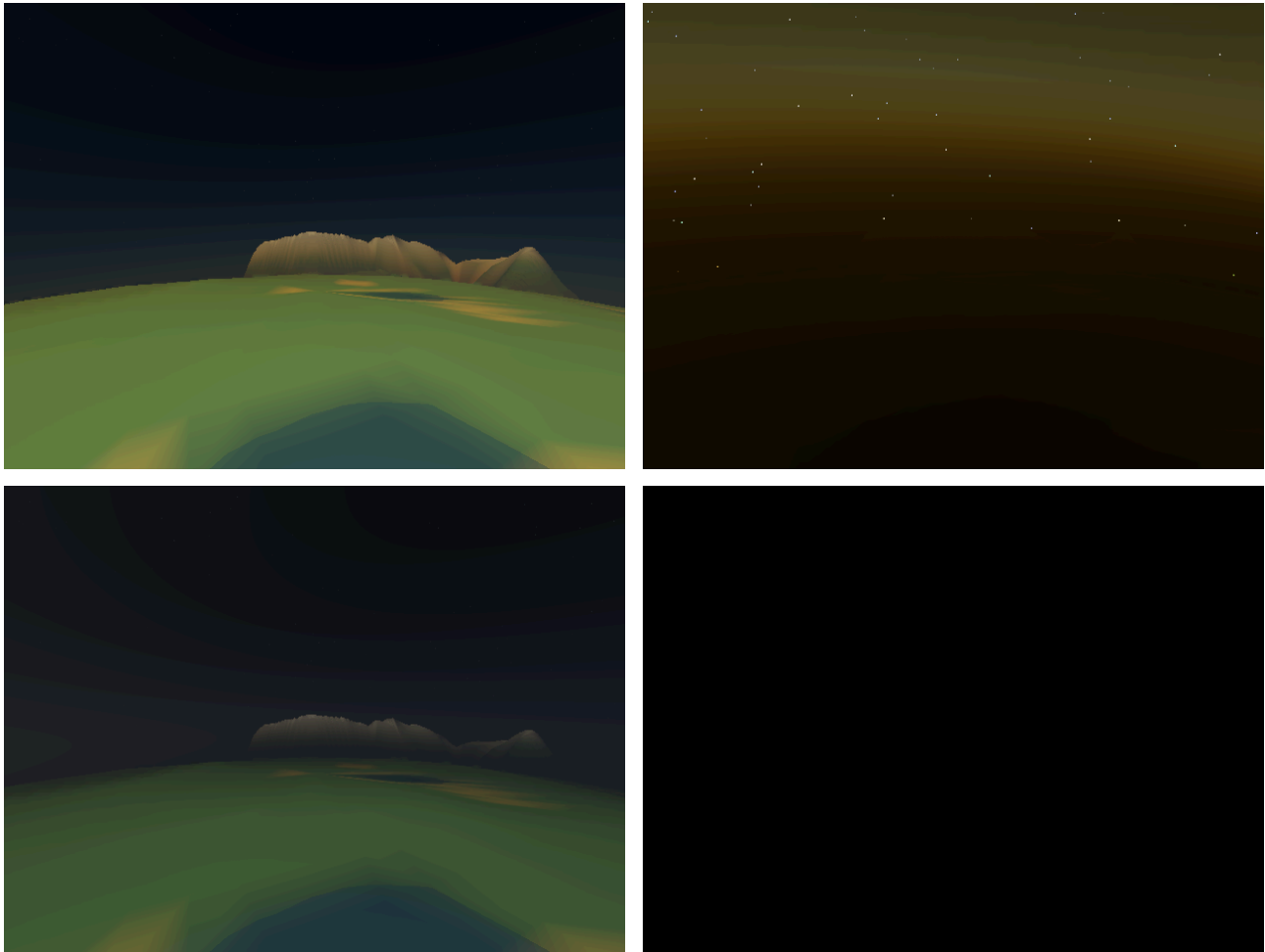


Figure 11: Current controlled atmospheric scenarios, left to right and top to bottom: standard air, sunset, condensed mist, and heavy red dust. All four are actual renderer captures from the regression gallery.

5.2. Reducing atmospheric cost without changing its data contract

The first atmosphere was visibly correct but expensive. `fa1a294` added a reusable GPU density-column table, reduced-resolution scattering fields, depth-aware resolve, paused-frame reuse, and GPU pass profiling. Shell-dependent columns need not be regenerated for camera motion or a changed gas mixture. The main atmosphere field normally uses quarter resolution; when reduced samples do not match full-resolution depth, the pixel is integrated at full resolution. Terrain, stars, and the Sun remain full resolution. The resulting pipeline is summarized in Figure 12.

Atmospheric composition: reusable columns and depth-aware fields

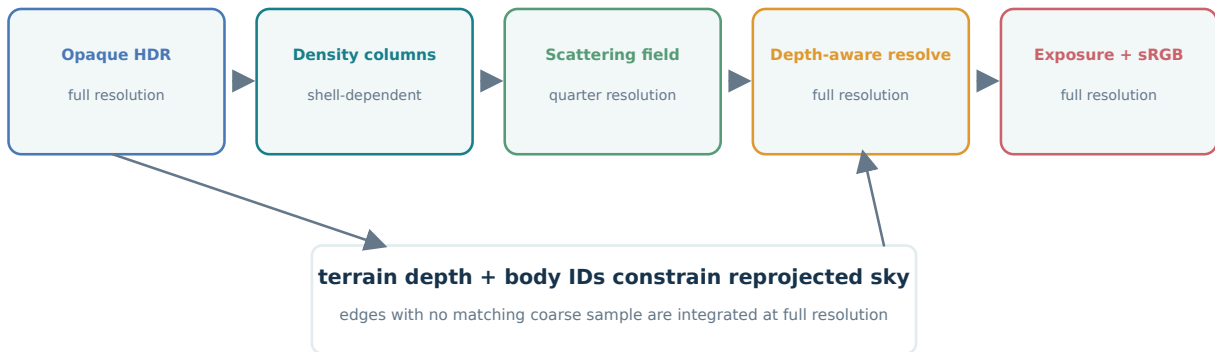


Figure 12: Atmospheric rendering graph. A reusable density-column lookup and a reduced scattering field lower repeated work; full-resolution depth controls edge reconstruction. Generated schematic.

The project’s recorded Quadro M1000M benchmark at 1280×720 reported 208 ms mean frame time before optimization, 138 ms after the column lookup, and 26 ms after reduced fields and shared ray calculations. These were 90-frame measurements of one surface scene with Debug CPU builds, recorded in the README; Figure 13 reproduces those historical values rather than claiming a new experiment. The approximately eightfold change is specific to that workload and GPU. A later RelWithDebInfo run reported 28.5 ms. A separate headless software-rendered capture is not comparable to this hardware benchmark.

Historical Quadro M1000M benchmark reported in the project README

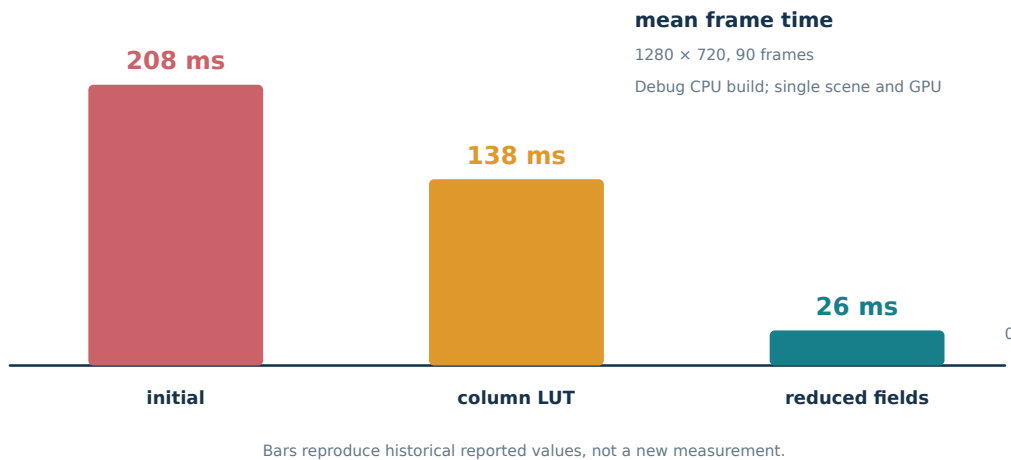


Figure 13: Historical mean frame times from README.md, Quadro M1000M, 1280×720 , 90 frames. These bars are a visualization of previously reported measurements, not a new benchmark.

A profiler now records CPU stage times and asynchronous GPU timestamp results without blocking for each query. CPU frame time, GPU pass time, and swap/presentation delay answer different questions; adding CPU and GPU times would double-count overlap. A completed HDR scene may be reused if camera, time, viewport, geometry, and pending-terrain state match. The HUD remains separately drawn so input can reveal or hide it immediately.

6. 28 September: observability, quality control and correctness

6.1. Diagnostics, dependency boundaries and adaptive quality

The diagnostics work [ab59069](#) added optional Linux NVIDIA NVML utilization to the existing I/Tab panel and organized source, shaders, tests, screenshots, and replay metadata by subsystem. Utilization is sampled off the render thread and is device-wide, while GPU pass milliseconds belong to the application’s measured passes.

Software rendering and ambiguous GPU matching show N/A; a fake driver library tests the ABI path without claiming validation on NVIDIA hardware. Figure 14 is the real captured panel.

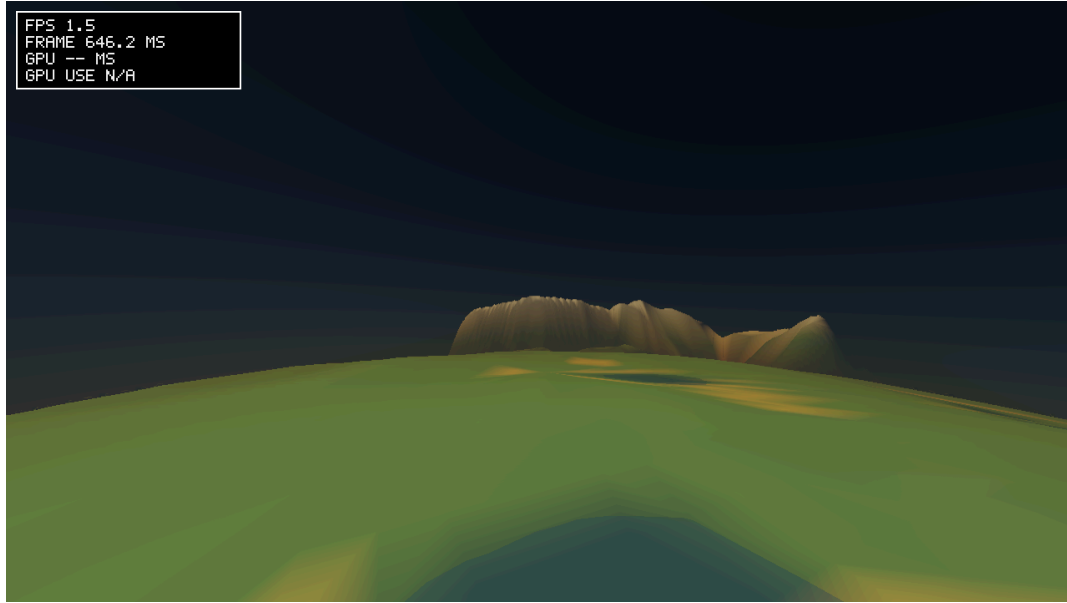


Figure 14: Actual current performance overlay under Xvfb/Mesa llvmpipe. The white-bordered black panel reports FPS, frame time, GPU pass time, and utilization availability; its numbers are capture-environment observations.

5c08479 implemented a best-effort 20 FPS quality target. A smoothed frame estimate $m_t = 0.85m_{t-1} + 0.15f_t$ selects one of six scene-resolution scales, from 100% down to 25%. After a 1.5-second cooldown, sustained times above 55 ms lower resolution; times below 32 ms permit recovery. Only moving interactive scenes feed the controller. A graphics-memory estimate can cap the initial scale, reserving most reported memory for other uses; fixed-resolution captures remain deterministic. The HUD stays at native window resolution. Figure 15 shows the control law, not a measured response trace. A strict 20 FPS guarantee is impossible once the minimum scale is reached or scene cost is unrelated to pixel count.

Best-effort quality control: timing feedback and memory cap

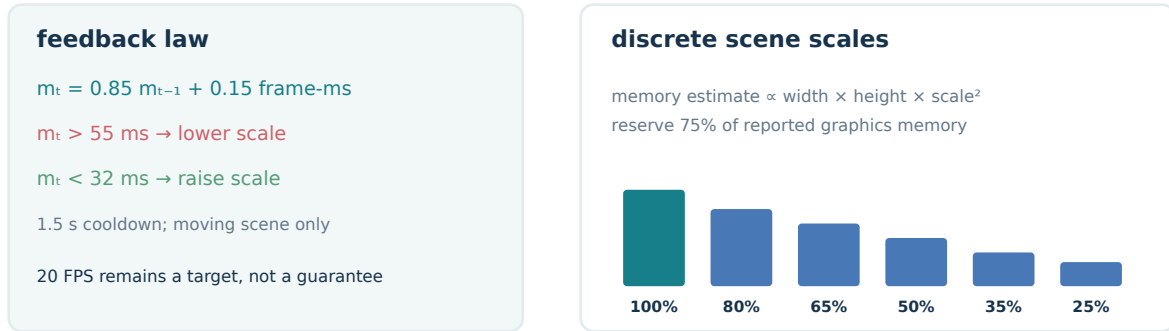


Figure 15: Actual controller thresholds and discrete scene scales; bar heights are relative scale, not measured frame rates. Generated schematic.

6.2. Geometric and illumination regressions

The later fixes address failures that a plausible screenshot could conceal. 1eafe2b corrected polar walking: numerically carrying a latitude/longitude heading through a pole reverses the local north/east basis, making the camera spin. The new walk step moves on a great circle and parallel-transport view and up about the axis $u \times t$ by angle $\frac{\text{distance}}{\text{radius}}$. The transported basis is then expressed in the new tangent frame. Figure 16 illustrates the singularity avoided by this construction.

Great-circle camera transport across a pole

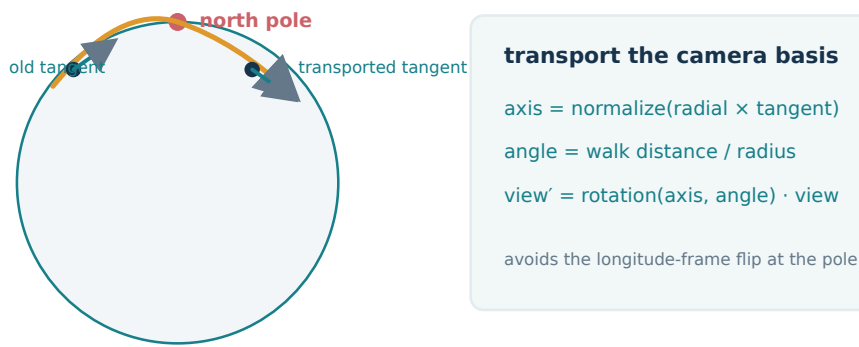


Figure 16: Great-circle position update and basis transport across the north pole. At the pole, a fresh longitude-derived frame can flip; a transported 3-D basis remains continuous. Generated schematic.

47394a7 reused each planet's terrain depth map inside direct atmospheric view-path scattering, so a mountain can block sunlight from nearby air. The finite solar disk's twilight region is retained, and indirect atmospheric fill remains. 73ce15f addressed water on a planet's shadow side: a valid reflection texture coordinate alone is insufficient evidence of sunlight. The reflection term is gated by local positive Sun incidence and terrain visibility, while indirect water radiance remains. Figure 17 illustrates the separation. The gallery's terrain-shadow and twilight cases are executable visual regressions rather than uncalibrated beauty shots.

Why a geometric reflection must be gated on the night side

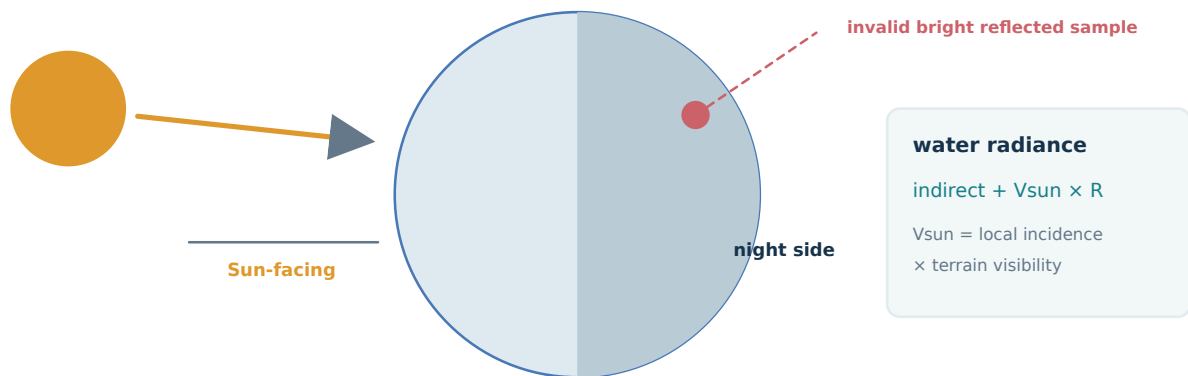


Figure 17: A reflected bright sample can exist in image space even when the water point lies on the night side. Local incidence and shadow-map visibility gate the direct reflection term. Generated schematic.

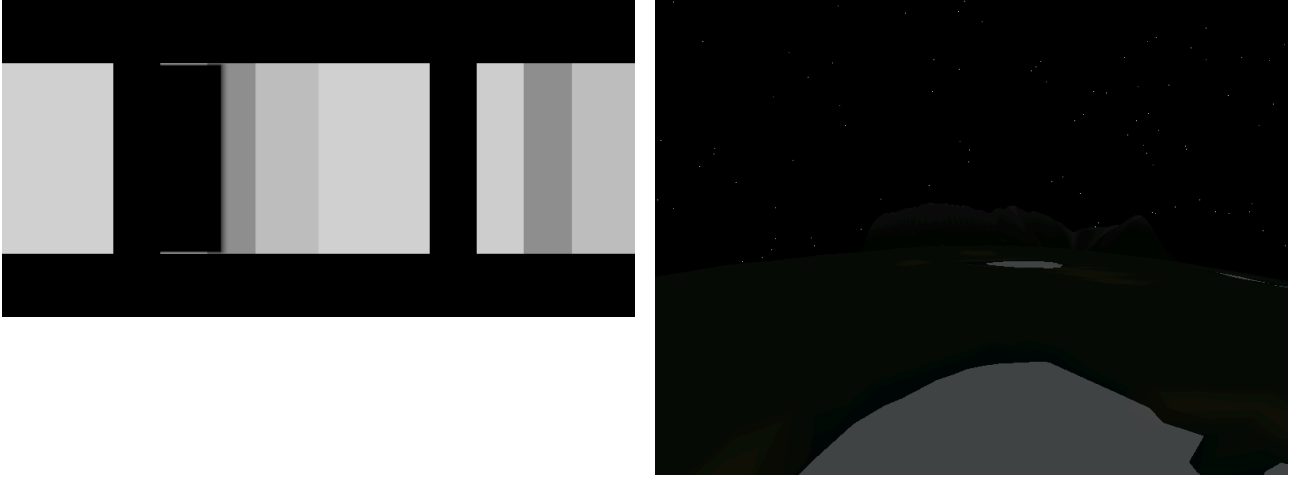


Figure 18: Actual current regression captures for terrain shadow visibility (left) and near-horizon twilight (right). Their fixtures and generation commands are recorded with the gallery.

6.3. Explanatory overlays and temporal camera continuity

8cd59ef introduced ten-revolution predictive trails and body cards. The path color averages terrain vertex RGB factors multiplied by the body's configured base tint; labels report name, parent, axes, eccentricity, period, radius, and mass. Sampling the entire orbital system, rather than each child's local ellipse alone, is necessary for moving-parent trails. The overlay is toggled by 0 and hidden in planet and surface cameras; real X11 input tests verify both behaviors.

Finally, 3e87f68 made orbit-to-surface entry a one-second wall-clock transition. The starting pose is the last actually rendered view, avoiding a first-frame jump after the key event. An eased parameter $q(u) = 3u^2 - 2u^3$ drives spherical interpolation of the planet-relative radial direction, logarithmic interpolation of radius, quaternion interpolation of orientation, and field-of-view interpolation. Intermediate positions are clamped above sampled terrain and water. The target is resampled while the planet moves, so the transition ends at the current surface pose rather than a stale one. Surface walking and cursor capture begin after the descent; switching modes cancels it. Figure 19 shows the path and ease curve.

One wall-clock second from orbital pose to surface pose

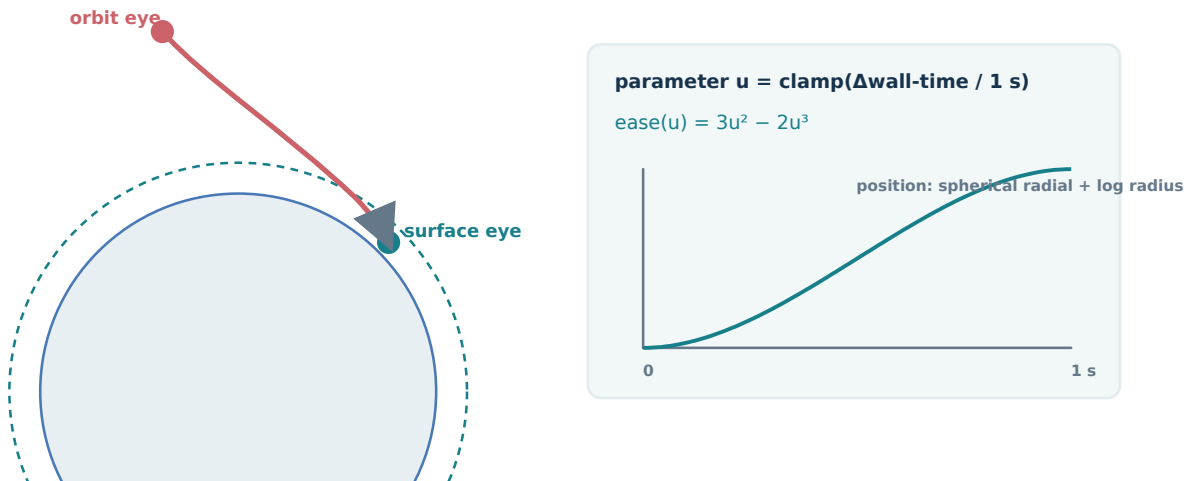


Figure 19: Orbit-to-surface descent. The curve shows the implemented smoothstep parameter; the spatial path is schematic because the actual start, target, and terrain vary by scene.

7. Evaluation, limitations and reproducibility

7.1. What the evidence establishes

The final tree at 9661b8e contains 39 CTest entries. The suite covers configuration and orbital invariants; terrain and camera behavior; lighting, shadows, atmosphere and refraction on OpenGL; renderer captures; adaptive quality; and real GLFW input under Xvfb. In the development environment, all 39 passed under Mesa llvmpipe. The 12 README images were regenerated for version 0.0.1 and checked against their SHA-256 manifest. These

facts establish that the specified examples and regressions work on the tested software renderer. They do not establish physical fidelity, portability to every GPU driver, or a guaranteed 20 FPS for arbitrary scenes.

The main model boundaries are deliberate. Orbits are prescribed two-body ellipses with hierarchical recoil, not numerically integrated mutual gravitation. Terrain is a deterministic synthetic height field with bounded tessellation; it is not geophysical erosion. Refraction bends view rays and reprojects available distant imagery but keeps near opaque geometry and direct shadow rays straight. Atmospheric temperature is read from JSON rather than evolved from sunlight and orbit. Device utilization is unavailable without a compatible, unambiguous NVIDIA driver. These constraints are important when interpreting visual results.

7.2. Rebuilding the paper and the evidence

From the repository root, regenerate the deterministic vector figures and compile the Typst source:

```
python3 docs/journal/make_figures.py
typst compile --root . --creation-timestamp 1790607816 docs/journal/paper.typ docs/journal/paper.pdf
```

The 13 SVGs require only Python’s standard library. The fixed PDF creation timestamp is the final implementation commit’s timestamp, so repeated builds do not encode the compilation time. The paper references the versioned PNGs already stored in `docs/screenshots/`. To rerun the application tests in a headless OpenGL environment, build the CMake project with testing enabled and execute `LIBGL_ALWAYS_SOFTWARE=1 xvfb-run -a ctest --test-dir build --output-on-failure`. Exact image-generation commands and checksums are in `docs/captures/generation.json`.

8. Conclusion

The chronological record shows a recurring pattern: a visual capability entered the renderer, a representation or performance boundary became visible, and a testable correction clarified that boundary. The result is a compact but coupled planetary system in which geometry, motion, lighting, atmosphere, quality control, and camera semantics are coordinated by explicit data contracts. The most transferable techniques are the separation of a stable sampled surface from its adaptive mesh, absolute-time evaluation of hierarchical orbits, linear-light reuse of visibility information, and nonblocking instrumentation that informs rendering decisions without becoming the workload being measured.

9. References and artifact trail

- R1. NASA, *Basics of Space Flight, Chapter 3: Gravity and Mechanics*, official orbit primer. Used for the classical two-body context; the implementation details above come from `src/simulation/OrbitalSystem.h`.
- R2. E. Bruneton and F. Neyret, “Precomputed Atmospheric Scattering,” *Computer Graphics Forum* 27(4), 2008, doi:10.1111/j.1467-8659.2008.01245.x. The project uses a smaller runtime single-scattering approximation and a limited reusable lookup.
- R3. NIST Engineering Metrology Toolbox, Index of Refraction of Air documentation. Background for pressure, temperature, humidity and refractive index; this project does not claim NIST-grade refractometry.
- R4. IAU 2015 resolutions, official PDF. The rendering code uses the Resolution B2 bolometric zero-point convention for luminosity.
- R5. *PlanetSimulation* repository, `git log --reverse` through 9661b8e; implementation in `src/`, shaders in `shaders/`, checks in `tests/`, and captured images with reproducibility metadata in `docs/captures/generation.json`.

10. Appendix: chronological anchors and evidence map

The table lists selected commits in ascending date order. It is an index to the source history, not a replacement for the intervening changes. Commit hashes identify the implementation states discussed in the paper.

Date	Commit	Historical anchor
09 Sep	bd73ba9	Structured scenario configuration
11 Sep	8246a31	Shader and mesh pipeline
11 Sep	94b42f6	Headless PNG render test
15 Sep	ee48919	GLM replaces custom matrix/vector stack
15 Sep	3d73170	Procedural planet generation works
24 Sep	18ffbad	Hierarchical absolute-time orbits
24 Sep	01c0b70	Reflected moonlight
24 Sep	96e2014	Terrain shadow maps
26 Sep	0e48155	Atmospheric model and renderer
26 Sep	fa1a294	Atmospheric performance work
28 Sep	ab59069	Diagnostics and subsystem organization
28 Sep	5c08479	Adaptive scene resolution
28 Sep	1eafe2b	Pole-stable camera transport
28 Sep	47394a7	Atmospheric terrain occlusion
28 Sep	73ce15f	Night-side ocean correction
28 Sep	8cd59ef	Predictive orbit overlay
28 Sep	3e87f68	One-second camera descent
28 Sep	9661b8e	JSON temperature refraction regressions

10.1. Claim-to-artifact map

Claim	Primary implementation	Validation
Stable terrain under LOD	src/rendering/geometry/Terrain.h	TerrainTests, surface capture
Barycentric orbit hierarchy	src/simulation/OrbitalSystem.h	OrbitalSystemTests
Shadowed atmosphere	shaders/atmosphere/atmosphere.frag	TerrainShadowRenderIntegration
Temperature-controlled bend	src/simulation/Atmosphere.h	AtmosphereTests, GPU render test
Adaptive scene scale	src/rendering/diagnostics/AdaptiveQuality.h	AdaptiveQualityTests, live render
One-second descent	src/rendering/camera/CameraTransition.h	CameraTransitionTests, X11 input